Numpy

Driss MATROUF, Professeur UAPV, 2025-2026

Numpy :Numerical Python

- Une bibliothèque (open source) qui est largement utilisée dans les sciences de l'ingénieur.
- Cette bibliothèque traite et manipule des tableaux de données multi-dimensions.
- Après avoir installer Numpy, vous pouvez l'utiliser en important la bibliothèque dans votre programme.

```
import numpy as np
```

- Pour créer un tableau de d'entiers

```
a = np.array([[3,2,3],[4,5,6]])
print(a.shape)
```

3	2	3
4	5	6

- Ceci affichera : (2,3)
- Pourquoi : améliorer la vitesse, optimiser l'utilisation de la mémoire
- Dans l'exemple il s'agit d'un tableau 2-D, généralement on peut manipuler des tableau N-D
- La classe utilisée est : `ndarray,` elle représente des tableau à N dimensions

- Créer un ndarray à partir d'une liste:
```
a=np.array([1,2,3,4,5,6])
print(a.shape)# ceci affichera (6,)
```

- `a[0]=10 # a est mutable`

- Notation Slice:
```
b=a[:3]
print(b) # affiche [10,2,3]
b=a[3:]
print(b) # affiche [4,5,6]
b[0]=40
print(a) # affiche [10,2,3,40,5,6]
a=np.array([1,2,3,4],[5,6,7,8],[9,10,11,12]]
print(a.shape) # affiche (3,4)
print(a[1,3]) # affiche 8
```

- Transposée
```
B = A.T
B = np.transpose(A)
```

- Attention : dans le cas du Slice et de la transposée, il y a partage de données entre B et A

- Si on veut que B et A ne partagent rien : `B=np.array(A.T)`

La slice

- Elle permet de sélectionner une partie d'un tableau (array)
- Une ligne ou une clonne peut être remplacée par : start:stop:step

```
start : indice de départ (inclus), par défaut = 0
stop : indice de fin (exclu), par défaut = la valeur max
step : pas entre les éléments (optionnel)
```

- Exemple 1 dimension :

```
a = np.array([10,20,30,40,50])
a[1:4] ==> [20, 30, 40]
a[:3] ==> [10, 20, 30]
a[2:] ==> [30, 40, 50]
a[::,2] ==> [10, 30,50]
```

- Exemple 2 dimensions:
  A=np.array([[1,2,3],[4,5,6],[7,8,9]])
  print(A.shape) ==> (3,3)
  A[0:2,1:3] ==> [[2,3],[5,6]]
  B= A[0:2,1:3] ==> Attention A et B partagent les mêmes données

Certains attribues

- ndim : le nombre de dimensions
```
print(a.ndim) # affiche 2
```

- shape : la forme (nbre de lignes, nbre de colonnes)
```
print(a.shape) # affiche (3,4)
```

- Changement de forme
```
# b est un nouveau objet
b=np.reshape(a, (2, 6)) #attention a et b partagent les mêmes données
# a change de forme sur place
a.reshape((6,2))
```

- size : la taille totale en nombre d'éléments
```
print(a.size == math.prod(a.shape) # affiche True
```

- dtype : le type des éléments du tableau
```
print(a.dtype) # affiche int64
```

Les tableaux de base

- ```
  np.zeros(2) # [0. , 0.]
  np.zero((2,3)) # array([[0., 0., 0.], [0., 0., 0.]])

  np.ones(2) # [1. , 1.]
  np.ones((2,3)) # array([[1., 1., 1.], [1., 1., 1.]])

  np.arange(4) # array([0, 1, 2, 3])
  np.arange(2, 9, 2) # array([2, 4, 6, 8])

  np.linspace(0, 10, num=5) # array([0. ,2.5, 5., 7.5, 10.])
  ```

- Le type de données est par défaut np.float64, mais nous pouvons le spécifier lors de la création:

  ```
  np.ones(2, dtype=np.int64)# array([1, 1])
  ```
  ```

Quelques fonctions

- **np.mean(…)**

```
>> import numpy as np
>> a = np.array([[1, 2], [3, 4]])
>> np.mean(a)
2.5
>> np.mean(a, axis=0)
array([2., 3.])
>> np.mean(a, axis=1)

import numpy as np

def moy(a):
 s=0
 for i in range(a.shape[0]):
  for j in range(a.shape[1]):
   s += a[i,j]
 return(s/a.size)

a=np.array([[1,1,1],[2,2,2]])
print(moy(a))

>> np.mean(a , axis=0)
array([2., 3.])
>> np.mean(a , axis=0)
array([1.5,3.5])
```

- numpy.max, numpy.min, numpy.sum, numpy.std

```
>>a = np.array([[0,1],[2,3]])

>>np.max(a)
3

>> np.max(a,axis=0)
array([2,3])

>>np.max(a,axis=1)
array([1,3])

>>data = np.array([10,15,29,25,30])

>>std = np.std(data)

>>print(" Ecart-type :", std)
Ecart-type : 7.07106791
```

Tableau aléatoires

- On peut créer un générateur :

```
>> rng = np.random.default_rng()
>> rng.random()
# Generate one random float uniformly
# distributed over the range [0, 1]
>> rng.standard_normal(10)
array([-0.310, -1.8922, -0.3628, -0.6352,  0.4318,0.5164,
1.2569,  0.0777,  0.8409, -2.1340])
# Generate an array of 10 numbers according to a unit
# Gaussian distribution

>>rng.integers(low=0, high=10, size=5)
# Generate an array of 5 integers uniformly over the range [0, 10)
>>rng.integers(low=0, high=10, size=(2,5))
array([[6, 5, 4, 0, 8],
     [8, 2, 9, 5, 6]])
```

Les distribution de probabilités

`binomial`(n, p[, size])
Draw samples from a binomial distribution.
`chisquare`(df[, size])
Draw samples from a chi-square distribution.
`dirichlet`(alpha[, size])
Draw samples from the Dirichlet distribution.
`exponential`([scale, size])
Draw samples from an exponential distribution.
`f`(dfnum, dfden[, size])
Draw samples from an F distribution.
`gamma`(shape[, scale, size])
Draw samples from a Gamma distribution.
`geometric`(p[, size])
Draw samples from the geometric distribution.
`gumbel`([loc, scale, size])
Draw samples from a Gumbel distribution.
`hypergeometric`(ngood, nbad, nsample[, size])
Draw samples from a Hypergeometric distribution.
`laplace`([loc, scale, size])
Draw samples from the Laplace or double exponential distribution with specified location (or mean) and scale (decay).
`logistic`([loc, scale, size])
Draw samples from a logistic distribution.
`lognormal`([mean, sigma, size])
Draw samples from a log-normal distribution.
`negative_binomial`(n, p[, size])
Draw samples from a negative binomial distribution.

`noncentral_chisquare`(df, nonc[, size])
Draw samples from a noncentral chi-square distribution.
`noncentral_f`(dfnum, dfden, nonc[, size])
Draw samples from the noncentral F distribution.
`normal`([loc, scale, size])
Draw random samples from a normal (Gaussian) distribution.
`pareto`(a[, size])
Draw samples from a Pareto II (AKA Lomax) distribution with specified shape.
`poisson`([lam, size])
Draw samples from a Poisson distribution.
`power`(a[, size])
Draws samples in [0, 1] from a power distribution with positive exponent a - 1.
`rayleigh`([scale, size])
Draw samples from a Rayleigh distribution.
`standard_cauchy`([size])
Draw samples from a standard Cauchy distribution with mode = 0.
`standard_exponential`([size, dtype, method, out])
Draw samples from the standard exponential distribution.
`standard_gamma`(shape[, size, dtype, out])
Draw samples from a standard Gamma distribution.
`standard_normal`([size, dtype, out])
Draw samples from a standard Normal distribution (mean=0, stdev=1).
`standard_t`(df[, size])
Draw samples from a standard Student's t distribution with df degrees of freedom.
`triangular`(left, mode, right[, size])
Draw samples from the triangular distribution over the interval `[left, right]`.
`uniform`([low, high, size])
Draw samples from a uniform distribution

Concaténation

- ```
  import numpy as np
  # Deux tableaux d'origine

  a = np.array([1, 2, 3])
  b = np.array([4, 5, 6])

  # Utilisation de concatenate
  c = np.concatenate((a, b))

  print("Tableau a :", a)  # [1, 2, 3]
  print("Tableau b :", b)  # [4, 5, 6]
  print("Tableau c :", c)  # [1, 2, 3, 4, 5, 6]

  # Vérifions les identifiants (adresses mémoire)
  print(id(a))  # Adresse mémoire de 'a'
  print(id(b))  # Adresse mémoire de 'b'
  print(id(c))  # Adresse mémoire de 'c' (différente)
  ```

- `c` est un objet distinct de a et de b
  ```

Indexing and Slicing

- `data = np.array([1, 2, 3])`
- `data[0:2] # [1,2]`
- `data[1:] # [2,3]`
- `data[-2:] # [2,3]`
- `a = np.array([[1, 2 ,3 ,4],[5, 6, 7, 8],[9, 10, 11, 12]])`
- `print(a[a < 7]) # [1, 2, 3, 4, 5, 6]`
- `five_up = (a >= 5)`
  `# [[False, False, False, False],[False, True, True, True], [True, True, True, True]]`

Indexing and Slicing

- `divisible_by_2 = a[a%2==0] # [ 2  4  6  8 10 12]`

- 2 conditions à la fois
  `c = a[(a > 2) & (a < 11)] # [ 3  4  5  6  7  8  9 10]`

- `five_up=(a>5) |(a == 5)`
  ```
  [[False False False False]
   [True  True  True  True]
   [True  True  True True]]
  ```

- ```
  a = np.array([[1, 2, 3, 4],[5, 6, 7, 8],[9, 10, 11, 12]])
  b = np.nonzero(a % 2 == 0)
  prin(b) # (array([0,0,1,1,2,2]),array([1,3,1,3,1, 3]))
  ```

- Ci-dessus chaque tableau dans le résultat contient un indice dans une dimension où la condition est vraie. On peut retrouver les valeurs elle même:

  `print(a[b]) # [2,4,6,8,10,12]`
  ```

- ```
  a1 = np.array([[1, 1],[2,2])
  a2 = np.array([[3, 3],[4,4])
  ```

- ```
  np.vstack((a1, a2))
  array([[1, 1],
         [2, 2],
         [3, 3],
         [4, 4]])
  ```

- ```
  np.hstack((a1, a2))
  array([[1, 1, 3, 3],
         [2, 2, 4, 4]])
  ```

- ```
  a = np.array([[1, 2, 3, 4],[5, 6, 7, 8],[9, 10, 11, 12]])
  b1 = a[0, :]
  print(b1) # 1, 2, 3, 4]
  b1[0] = 99
  print(a) # array([[99,2,3,4],[5,6,7,8],[9,10,11,12]])
  ```

- ```
  b2 = a.copy() # b2 est une copie de a physiquement indépendante
  ```
  ```

- `+, -, * : permet l'addition, la soustraction la multiplication de deux ndarray (élément à élément)`

- ```
  a=np.array([1,2,3])
  a = a - 1 # soustraire 1 à tous les éléments de a
  a = a * 1.6 # multiplier par 1.6 tous les éléments de a
  ```

- ```
  a=np.array([1,2,3],
             [1,2,3])
  b=np.array([1,1,1])
  a + b
  array([[2,3,4],
         [2,3,4]])
  ```

- ```
  a.sum()
  12
  ```

- ```
  a.sum(axis=0)
  array( [2,4,6])
  ```

- ```
  a.sum(axis=1)
  array([6,6])
  ```

- Les utilisations de `min, max, mean, var, std` sont similaires à `sum`
  ```

Multiplication

- `C = A @ B`
  ```
  A=np.array([[1,2],[3,4]])
  B=np.array([[5,6],[7,8]])
  C = A @ B
  print(C)
  [[19 22]
     [43 50]]
  ```

shapes	résultat
`(m,n) @ (n,p)`	`(m,p)`
`(m,n) @ (n,)`	`(m,)`
`(n,) @ (n,p)`	`(p,)`
`(m,n) @ (k,p)` avec `n ≠ k`	erreur

- `v = np.array([1,2]) ==> v.shape=(2,) # v est un vecteur 1D`
  On peut faire : `A @ v` # v est considéré comme un vecteur colonne
  On peut faire : `v @ A` # v est considéré comme un vecteur ligne

- `A @ v ==> array([ 5, 11])`

- `v=v.reshape(2,1) ==> A @ v ==> array([[5],[11]]) # shape=(2,1)`

- `C = np.matmul(A,B) # la même chose que @`

- `C=np.dot(A,v) ou C=A.dot(v) # la même chose que @`

L’opérateur *

- Multiplication élément à élément : produit de Hadamard.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 & 7 \\ 6 & 8 \end{bmatrix} ==> \begin{bmatrix} 1 \times 5 & 2 \times 7 \\ 3 \times 7 & 4 \times 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * 2 ==> \begin{bmatrix} 1 \times 2 & 2 \times 2 \\ 3 \times 2 & 4 \times 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 & 7 \end{bmatrix} ==> \begin{bmatrix} 1 \times 5 & 2 \times 7 \\ 3 \times 5 & 4 \times 7 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 \\ 7 \end{bmatrix} ==> \begin{bmatrix} 1 \times 5 & 2 \times 5 \\ 3 \times 7 & 4 \times 7 \end{bmatrix}$$

- Exercice: Je veux une matrice :

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 2 & 3 \\ 0 & 1 & 2 & 3 \end{bmatrix} \text{pour ce faire,} \quad \overset{\mathbf{A}}{\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}} * \overset{\mathbf{B}}{[0 \quad 1 \quad 2 \quad 3]}$$

```
A = np.ones((6,4))
B = np.arange(4) # [0,1,2,3]
A*B
```

- `(m,n)*(m,n);(m,n)*scalaire;(m,n)*(n,),(m,n)*(m,1)`

- Exercice: écrire un programme qui génère aléatoirement une matrice (n,m) et qui calcule la moyenne sur les lignes sans utiliser la fonction mean, mais seulement des produits matriciels.

```
import numpy as np
# dimensions
m = 10
n = 5

# génération d'une matrice aléatoire
A = np.random.rand(m, n)

# vecteur de uns
u = np.ones(m)

# moyenne des lignes par produit matriciel
moyennes = (u @ A) / m

print("Matrice A :")
print(A)

print("\nMoyenne des lignes :")
print(moyennes)

# pour vérifier:

print(np.m
```

- Ecrire une fonction qui transforme une matrice de nombres positifs en une matrice stochastique. Une matrice est dite stochastique si la sommes des valeurs qui se trouvent sur chaque colonne est égal à 1.

```
import numapy as ns np

def matrice_stochastique(A):
      sommes = np.sum(A,axis=0)
      Return A/sommes

def matrice_stochastique(A):
      sommes = np.sum(A,axis=0)
      A/=sommes

def matrice_stochastique(A):
      sommes = np.sum(A,axis=0)
      A = A/sommes
```

Résolution de système linéaire

- Comment calculer le rang : utiliser **numpy.linalg.matrix_rank(…)**

```
import numpy as np
A = np.array([[1,2,3],[2,2,2],[2,3,1]])
print (numpy.linalg.matrix_rank(A))
```

- Comment résoudre $Ax = b$ (système $n \times n$)

  1- Vérifier que le rang de la matrice est égale à n
  2- Ensuite utiliser **numpy.linalg.solve()**

```
A = np.array([[1,2,3],[2,3,5],[3,1,3]])
b = np.array([[1,1,1])
x = np.linalg.solve(A, b)
print(x)
[ 2.  4. -3.]
```